SLOWPOKE: End-to-end Throughput Optimization Modeling for Microservice Applications

Yizheng Xie
Brown University

Di Jin
Brown University

Oğuzhan Çölkesen
Brown University

Vasiliki Kalavri
Boston University

John Liagouris
Boston University

Nikos Vasilakis
Brown University

Abstract

We present SLOWPOKE, a causal profiling tool for microservice applications, that accurately quantifies the effect of hypothetical optimizations on end-to-end throughput, without relying on tracing or a priori knowledge of the call graph. Microservice operators can use SLOWPOKE to ask what-if performance analysis questions of the form *"What throughput could my retail application sustain if I optimize the shopping cart service from 10K req/s to 20K req/s?"*. Given a target service and optimization, SLOWPOKE employs a performance model that determines how to selectively slow down non-target services to preserve the relative effect of the optimization. It then performs profiling experiments to predict end-to-end throughput, as if the candidate optimization was implemented. When applied to four real-world microservice applications, SLOWPOKE accurately quantifies optimizations with a small mean absolute error of 2.07%. We show that SLOWPOKE is also effective in more complex scenarios, such as predicting throughput after scaling optimizations or when the bottleneck is caused by mutex contention. We further evaluate SLOWPOKE in a large-scale deployment with 45 nodes, as well as across 108 synthetic benchmarks, covering a wide range of microservice characteristics.

1 Introduction

The microservice architecture has emerged as a prevailing approach for constructing modern distributed applications [22,28,31,44,48,59]. By decomposing applications into independently developed and deployed services, microservices simplify cross-team collaboration and scalable deployment [21,28,35]. The throughput of an application is fundamental to its scalability, cost efficiency, and quality of service under high incoming traffic, e.g., traffic spikes during overlapping periods of global activity for social media platforms [18].

Improving the throughput of individual services, even with simple techniques like vertical or horizontal scaling, does not necessarily yield end-to-end throughput improvements, due to complex service interactions in real deployments [11]. As a result, estimating how the end-to-end throughput of a microservice architecture would change after optimizing one or more services remains a challenging open problem. The current practice is to implement selected optimizations and evaluate performance in a canary environment [41,47]; yet, this approach leaves the hard task of identifying the most promising changes to the engineering teams. *What-if* analysis provides a principled approach to address this problem by quantifying the effects of hypothetical optimizations ahead of time and with high accuracy. Example optimizations include allocating additional resources, improving runtime efficiency like multithreading, or migrating to new frameworks or cloud platforms; for instance, adopting a high-capacity data store in data-as-a-service infrastructures [27].

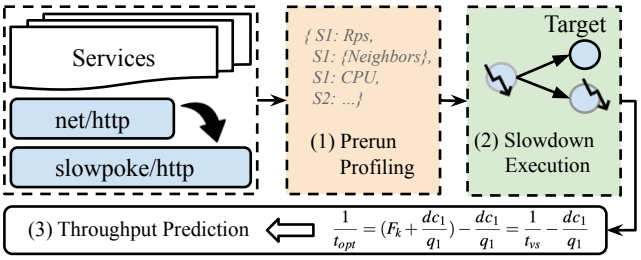


Figure 1: SLOWPOKE Overview. Given a microservice application running on Kubernetes, SLOWPOKE (1) pre-runs experiments to collect per-service information; (2) selectively slows down non-target services and measures the resulting throughput; and (3) quantifies throughput improvements using a performance model.

Existing what-if analysis techniques, such as causal profiling [17], quantify the impact of optimizing a specific section of code in short-running multithreaded applications on *a single machine*. The core idea behind causal profiling is to introduce *virtual speedups* by carefully pausing concurrent execution threads. A virtual speedup has the same relative effect as a hypothetical speedup and can be used to accurately estimate end-to-end performance before applying the

real optimization. Unfortunately, the state-of-the-art causal profiler, Coz [17], cannot be applied to the distributed setting. Coz assumes a single monolithic application and can only predict the effect of optimizing local functions by putting the application threads to sleep. On the other hand, based on distributed tracing [2, 19, 30, 45], existing microservice profiling tools [15,29,34,52,57,58] identify critical paths and predict latency. Without an accurate causal model and visibility into the services' internal states, these approaches cannot answer what-if questions about throughput, since there is no well-defined correlation between latency and throughput.

This paper presents SLOWPOKE, a system that accurately quantifies end-to-end throughput improvements of hypothetical optimizations. Inspired by causal profiling, SLOWPOKE introduces a performance model that estimates the throughput of a microservice application by carefully slowing down services using a lightweight distributed mechanism for coordinated virtual speedups. Given a microservice application, the target service for optimization, and a pre-defined workload, SLOWPOKE runs experiments in the *pre-production environment*. As shown in Fig.1, SLOWPOKE first collects necessary callgraph information in a prerun profiling stage, then computes and applies the appropriate virtual speedups to measure the end-to-end throughput, and lastly predicts the would-be throughput using SLOWPOKE's performance model.

We make the following contributions:

- A novel performance model for quantifying the end-to-end throughput improvements of hypothetical optimizations. Our model supports multithreaded services, dynamic call graphs, and hybrid workloads with different request types. We formalize the prediction problem using linear programming and demonstrate the equivalence between virtual speedups and actual speedups.
- SLOWPOKE, the first system for accurate what-if analysis of throughput optimizations in complex microservice architectures. SLOWPOKE treats microservices as black boxes communicating via HTTP or gRPC. SLOWPOKE does not require application instrumentation or a priori knowledge of the service call graph.
- A lightweight distributed slowdown mechanism that enables coordinated pausing across complex microservices, allowing SLOWPOKE to accurately quantify the effects of hypothetical optimizations at scale.

We evaluate SLOWPOKE on 4 real-world microservice applications and 108 synthetic benchmarks covering a wide spectrum of microservice topologies, execution models, and communication primitives. Our results show that SLOWPOKE provides accurate throughput predictions with errors ranging between -4.35–4.88% (R—root mean squared error: 2.07%) for real applications, and -7.61–5.65% (R: 1.89%) for synthetic benchmarks.

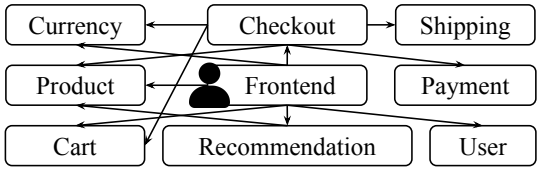


Figure 2: The topology of OnlineBoutique: vertices represent independently developed services exposing APIs. Edges denote various types of network communications between services.

SLOWPOKE is an MIT-licensed, open-source project available at https://github.com/atlas-brown/slowpoke.

2 SLOWPOKE Overview

In this section, we first discuss the challenges of achieving accurate what-if throughput analysis for microservice applications and present the motivation behind SLOWPOKE. We then provide an overview of SLOWPOKE's profiling capabilities and core system components.

2.1 Motivating Example

Consider the topology of the shopping application OnlineBoutique [3] shown in Fig. 2. To place an order or perform a product search, users send HTTP requests to the `frontend` service, which communicates with other services via network protocols, such as gRPC or HTTP. As demand scales—e.g., through market expansion—developers are often tasked with optimizing throughput to sustain performance under higher loads. Should the team accelerate `recommendation` queries by 30% through a costly model retraining, or improve `cart` lookups by 40% via implementing a high-capacity in-memory key-value store? Choosing the right optimization is critical to achieving meaningful end-to-end throughput gains within limited engineering and financial budgets.

Challenges and complexity: Predicting the end-to-end impact of a hypothetical optimization before implementing it presents significant challenges in complex microservice applications. For a given workload, assume each service S can sustain throughput t_S when other services are sufficiently fast. Throughput what-if analysis is reduced to accurately estimating $\min t_S$ before and after an optimization to a target service.

Given the microservice call graph, one approach is to estimate t_S *analytically*, by directly predicting which service would become the bottleneck after the optimization. However, this unknown throughput t_S depends on many factors, including deployment characteristics, the type of requests, the request rate, the service states, and the interactions with other services. Even with distributed tracing, the lack of visibility into the service internals prevents accurate estimation of t_S.

Another approach is to estimate t_S *experimentally*, by measuring the throughput of service S when it is saturated, which

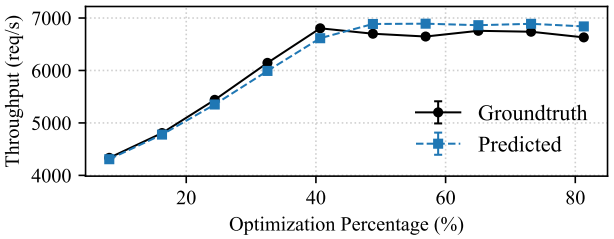


Figure 3: SLOWPOKE profiling result for the `Cart` service in OnlineBoutique with mixed types of user requests: the x-axis is the percentage of optimization on `Cart` service, 50% optimization means a 50% reduction in average execution time per request, the y-axis is the end-to-end application throughput.

is impractical. When deploying S as part of the entire system, to ensure no other services become a bottleneck first, it requires substantial resource over-provisioning to all other services when S is far from saturation. More importantly, provisioning more resources may not even increase throughput of a service as it can be single-threaded or bottlenecked by mutex contention. When deploying S in isolation, it requires capturing a substantial amount of end-to-end request traces and developing a workload driver capable of replaying both incoming calls and callbacks from downstream services to stress-test S, at arbitrary speeds. Unfortunately, even with a sophisticated replayer, changing the timescale of captured traces is error-prone, as it depends on the behavior of other services, which are unknown.

Alternatively, while one could, in principle, simulate an optimization by provisioning additional resources to the target service, this approach is often impractical given the aforementioned problems in resource provisioning. Furthermore, it fails to accurately capture the impact of other optimization types, such as algorithmic enhancements or migration to a more efficient execution framework.

2.2 What-if Analysis with SLOWPOKE

SLOWPOKE addresses the above challenges by performing profiling experiments that introduce *virtual speedups*, which have the same relative effect as the hypothetical speedups (see §3). As a result, it can accurately estimate the effect of hypothetical optimizations on the end-to-end throughput, without application-level instrumentation or a priori knowledge of the throughput capacity of individual services.

To perform what-if analysis, SLOWPOKE needs to run in a pre-production environment that mimics the characteristics of the production environment. Developers need to replace the communication libraries with SLOWPOKE's runtime and provide two inputs: (i) a target service and (ii) a workload consisting of end-user requests. Fig.3 shows an example profiling result of SLOWPOKE for the `Cart` service in OnlineBoutique. Without actually implementing any optimization, SLOWPOKE accurately quantifies the effect of optimizing the `Cart` service on end-to-end throughput. By selecting the point on the x-axis corresponding to the expected performance improvement of the target service (even when concrete optimizations are not yet implemented), developers can use the curve to immediately retrieve the predicted end-to-end throughput of the application. For example, using this specific profiling summary, developers can conclude that reducing `Cart`'s average processing time by 40% achieves the optimal throughput, and further optimization yields no additional benefits.

SLOWPOKE design: Fig. 4 presents an overview of SLOWPOKE's system design. SLOWPOKE targets microservice applications deployed within containerized environments managed by Kubernetes. Its core logic runs at the control plane of the orchestration platform. To leverage SLOWPOKE profiling, each service runs inside a container within a Kubernetes pod and communicates with other services via SLOWPOKE's communication libraries, instrumented with performance monitoring capabilities. SLOWPOKE's libraries serve as a drop-in replacement for existing HTTP and gRPC libraries, allowing developers to easily integrate SLOWPOKE into their infrastructure. In addition, SLOWPOKE deploys a lightweight controller program, called POKER, alongside each microservice instance within the same container pod. POKER is written in C and is responsible for collecting system information, applying virtual speedups determined by SLOWPOKE's performance model (§3), and communicating with neighboring POKER instances to achieve coordinated slowdown (§4).

SLOWPOKE operates in two phases. During the *pre-profiling* phase, it performs experiments to collect information about the deployment and request loads. It then feeds the collected information to its performance model (§3), which computes per-service virtual speedups. The performance model determines how much to selectively slow down other services, in order to produce a *bottleneck-equivalent* version of the optimized application. Bottleneck equivalence implies that the slowed-down execution will exhibit a bottleneck on the same service as the hypothetical optimized execution. In the *slowdown* phase, SLOWPOKE applies the virtual speedups using its coordinated slowdown mechanism (§4) and measures the throughput of the bottleneck-equivalent execution. At the end of this phase, SLOWPOKE computes the predicted end-to-end throughput, as if the candidate optimization was implemented, and produces profiling summaries like the one shown in Fig.3.

Experiment orchestration: SLOWPOKE extends Go's HTTP and gRPC libraries to enable non-intrusive service instrumentation and collection of system information. These extensions are API-compatible with the standard libraries, allowing developers to adopt SLOWPOKE with minimal integration effort (Fig. 4 (1)).

The instrumented libraries intercept handler invocations to record two runtime metrics per service: (i) the number of requests received, and (ii) its downstream services receiving

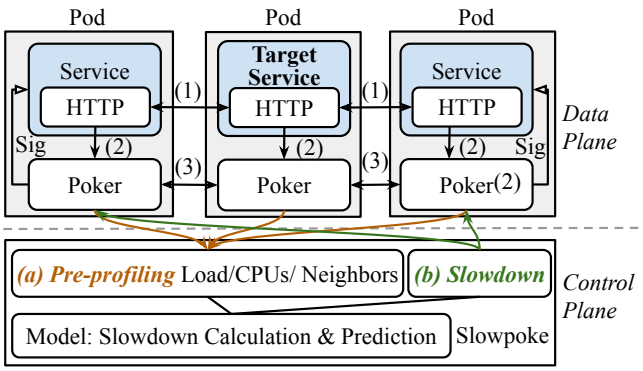


Figure 4: Architecture of SLOWPOKE. SLOWPOKE runs on the control node and orchestrates profiling experiments on microservice applications deployed on Kubernetes. Each microservice runs in a pod, alongside a POKER process. Services interact through instrumented networking libraries via a data channel (1). During the pre-profiling phase, POKER collects information from communication libraries through a standard Unix pipe (2). SLOWPOKE aggregates this information and computes the slowdowns using the performance model. During the slowdown execution phase, the POKER instance co-located with the target service sends slowdown messages to its neighboring POKERs through TCP control channels (3). Other POKERs will further propagate slowdown messages and pause the execution of non-target services using POSIX signals.

outgoing requests. The first metric is used in the pre-profiling phase to estimate the request distribution across services and, in the slowdown phase, to inform the local POKER instance about how much slowdown to inject (and when). The second metric is used to identify neighboring services and establish TCP connections between POKER instances, which are used for inter-POKER communication during the slowdown phase. POKER collects metrics from its co-located service's communication library through a standard Unix pipe (Fig. 4 (2)).

During pre-profiling, each POKER instance collects service-level metadata from the instrumented communication libraries, including load statistics, such as the number of calls per second, the directly-connected neighbor services, and the number of allocated CPUs. This information is aggregated by SLOWPOKE to compute slowdowns for each service, using its performance model. During the slowdown phase, SLOWPOKE restarts the deployment with slowdowns enabled by setting environment variables in the YAML configuration that inform POKER about the intended slowdown duration for its co-located service. When the target service (the one hypothetically optimized) receives a predefined number of incoming requests, its instrumented communication library notifies the local POKER instance (Fig. 4 (3)). POKER then initiates slowdown propagation by communicating with neighboring POKER instances. Upon receiving the slowdown messages, POKERs co-located with non-target services propagate them to their neighbors and pause the corresponding service processes by sending POSIX signals.

3 SLOWPOKE Performance Model

This section presents the mathematical model that SLOWPOKE relies on to quantify the end-to-end throughput improvement of an optimization to a target service by selectively slowing down non-target services. We first start with a simplified example for exposition (§3.1), then we provide a general model (§3.2) for more complex real-world applications.

3.1 Model Intuition

Consider the application with three services in Fig.5: A, B, and C. Let us assume the services are synchronous and CPU-bound—implying a reciprocal relationship between processing time and throughput—and each allocated with one CPU. As shown in Fig.5(a), each user request generates exactly one call to each service. A acts as the frontend: it receives user requests, processes them for one time unit, calls B, and waits for a response before replying to the user. Similarly, B processes the call for three time units and calls C. Lastly, C processes the call for another two time units, and returns. The application throughput is determined by the longest processing time among all services (*i.e.*, B):

$$t_{original} = \frac{1}{\max(p_A, p_B, p_C)} = \frac{1}{3} \tag{1}$$

where p_S is the average processing time per request in S.

Suppose an optimization plan optimizes target service B by reducing p_B by $d = 1$ time unit, as shown in Fig. 5(b). The application's throughput improves:

$$t_{opt} = \frac{1}{\max(p_A, p_B - d, p_C)} = \frac{1}{2} \tag{2}$$

However, this equation cannot be computed directly without knowing p_A, p_B, and p_C, which requires precise estimation of processing time. Rather than directly optimizing B, SLOWPOKE *virtually speeds up* target service by slowing down other services by d time units, e.g., one time unit in Fig.5(c). The resulting throughput is given by:

$$t_{vs} = \frac{1}{\max(p_A + d, p_B, p_C + d)} = \frac{1}{3} \tag{3}$$

Noticing that t_{opt} and t_{vs} are connected by $\max(p_A, p_B - d, p_C) = \max(p_A + d, p_B, p_C + d) - d$, therefore

$$\frac{1}{t_{opt}} = \frac{1}{t_{vs}} - d = 2 \tag{4}$$

Using this relationship, the model predicts the throughput after actual optimization by leveraging the *observed throughput* after artificially injecting slowdowns to non-target services, *without* requiring measurement of the throughput function of individual services or knowing the bottleneck.

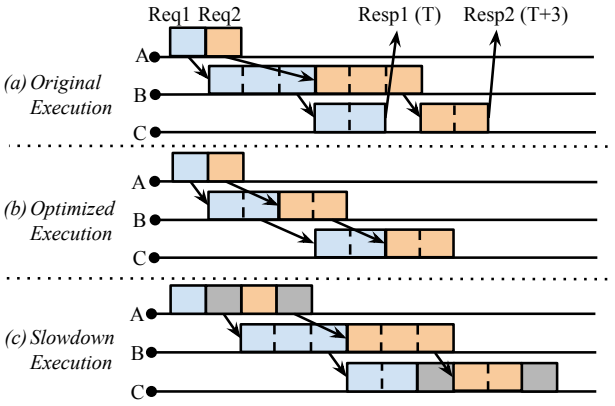


Figure 5: Each solid block represents the processing time of a service for a user request, *e.g.*, three time units for service B in (a). Blocks with the same color correspond to the same user request. Arrows represent network calls between services, triggered upon completion of the caller's processing block. For simplicity, network transmission latency is not shown. (a) Baseline execution graph: A request is completed and returned to the user every three time units. (b) Optimized execution: service B's processing time is reduced from three to two time units, resulting in improved overall throughput. (c) Slowed-down execution: an artificial delay of one time unit per request is introduced in services B and C. Gray blocks indicate the induced slowdown during service execution.

Table 1: Variables used in the model. RU can be any unit for the particular resource (*e.g.*, CPU time).

Var.	Description	Unit
t	End-to-end application throughput	Req/s
c	Number of calls per user request	1
q	Quota of a specific resource	RU/s
u	Resource consumed per call	RU/Req

3.2 Generalization

Here we show how the model generalizes to any type of resource that has a limit per time unit. Assume there are c calls to the service for each user request, each call takes u resource, and q is the quota for such resource. u can also be interpreted as the average resource consumption per call, given non-deterministic and non-cumulative per-request usage, *e.g.*, memory consumption varying with the input and cache state. For any feasible throughput t, the total use of the resource must not exceed the quota:

$$uct \le q \quad \text{or equivalently} \quad \frac{uct}{q} \le 1 \tag{5}$$

The units of the variables are shown in Tab.1. Each constraint inequality represents a single resource usage, and each service can have multiple of them. Perhaps unintuitively, a mutex-protected critical section can also be described in this fashion. Let u be the average time a service call spends inside the critical section, then by the nature of mutexes, uct needs to be less than 1, because no two threads can be in the critical section at the same time. In reality q may be slightly less than 1 because the locking may have some overhead. Nevertheless this formulation can describe the bottleneck of various synchronization primitives.

The end-to-end throughput is the maximal t that satisfies all the resource constraints. The throughput bottleneck is the resource whose usage constraint takes the equality sign. This linear programming problem is implicitly solved when measuring the end-to-end throughput of the application.

Dynamic call graphs and hybrid workload: The actual number of calls to each service varies based on application logic. One service may be called multiple times in response to a single user request. The service calls can also be dynamic: a service may decide to call or not call another service based on the request. Workload can also affect the number of calls a service receives. If a workload is a mix of two types of requests and only one of them uses the service, then the number of calls received depends on the ratio of the mixture.

The model encapsulates these behaviors by defining c as the average ratio between calls received over the number of user requests. We assume the distribution of request type remains steady throughout the entire workload.

Multi-core services and general resource quota: In Section 3.1 we assumed the services being single-threaded. In this formulation, we can relax that assumption by viewing CPU time as a resource. q becomes the CPU time the service gets per second — if 2 CPUs are dedicated to this service, $q = 2$. Accordingly, u becomes the average CPU time needed to handle each service call.

The concepts of quota and usage generalize to other types of resource as well. One notable example is mutex, in which case q is 1 CPU and u is time spent inside the mutex by each call to this service. This constraint will be *alongside* the regular CPU time constraint. Either constraint can become the service's bottleneck, and the linear programming problem correctly captures that.

Bottleneck-equivalent transformation: Assuming an optimization reduces usage u_1 of service 1 by d, the application throughput becomes the solution to the new constraint set:

$$\text{maximize } t \text{ in } \begin{cases} \dfrac{u_1 - d}{q_1} c_1 t \le 1 \\ \dfrac{u_i}{q_i} c_i t \le 1 \quad (i \ne 1) \end{cases} \tag{6}$$

For simplicity, this can also be written as

$$\begin{gathered} \text{maximize } t \text{ in } F_i t \le 1 \\ \text{where } F_1 = \frac{u_1 - d}{q_1} c_1 \text{ and } F_i = \frac{u_i}{q_i} c_i \quad (i \ne 1) \end{gathered} \tag{7}$$

In other words, given F_i as constants, the throughput of the optimized application t_{opt} is the solution to the linear programming problem in Eq.7. However t_{opt} cannot be obtained by solving this linear programming problem because u_i and q_i cannot be accurately known without extensive tracing instrumentations. It also cannot be obtained by measuring the application throughput because one cannot reduce the usage to $u_1 - d$ without actually implementing the optimization.

Similar to the transformation from Eq.2 to Eq.3, SLOWPOKE adds $\frac{dc_1 t}{q_1}$ to the left hand side of all the constraints, transforming the problem into

$$\text{maximize } t \text{ in } (F_i + \frac{dc_1}{q_1})t \leq 1 \tag{8}$$

The new linear programming problem does not have the same solution as the one in Eq.7. Yet, it does have the exact same bottleneck. Suppose the k-th constraint reaches equality in Eq.7, resource k will be the throughput bottleneck in the optimized application. Given t is the same in every constraint, it also means F_k is the largest of any F_i. Consequently, the left-hand-side of the k-th constraint in Eq.8 will also be the largest among all the constraints. Therefore Eq.8's k-th constraint also reaches equality. In other words, given the solution to Eq.8, t_{vs}, one can recover the solution to Eq.7, t_{opt}.

$$\frac{1}{t_{opt}} = (F_k + \frac{dc_1}{q_1}) - \frac{dc_1}{q_1} = \frac{1}{t_{vs}} - \frac{dc_1}{q_1} \tag{9}$$

Implicit solution through experiment: Expanding the definition of F_i in Eq. 8 we get

$$\text{maximize } t \text{ in } \begin{cases} \frac{u_1}{q_1} c_1 t \leq 1 \\ (\frac{u_i}{q_i} + \frac{c_1 d}{q_1 c_i}) c_i t \leq 1 \quad (i \neq 1) \end{cases} \tag{10}$$

While Eq.7 cannot be implicitly solved by throughput measurement experiment because of the negative term for constraint 1, Eq.8 can. By adding $\frac{dc_1}{q_1 c_i}$ seconds of "pause to resource consumption" for each service call, we can construct a version of the application whose throughput, once experimentally measured, implicitly solves Eq.8.

Measurement requirement: In the process of using the performance model, only d, q_1, and c_i needs to be known in order to calculate the appropriate delay and predict the throughput. Although u_i and q_i are used to model the problem, they are not required to be explicitly known, since they are implicitly factored in during the measurement.

4 Slowdown Mechanism

The mechanism to slow down other services is crucial to constructing the experiment for bottleneck-equivalent transformation. As pointed out in Section 3, a resource bottleneck

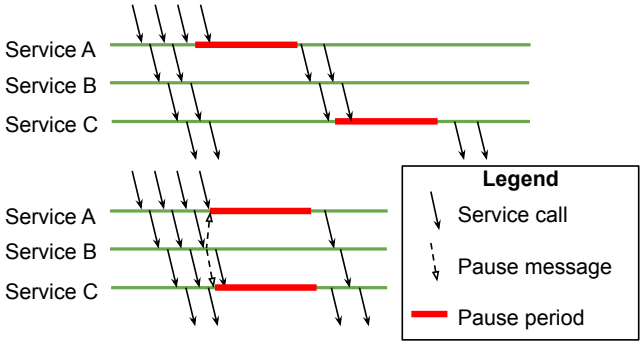


Figure 6: Uncoordinated (top) *vs.* Coordinated (bottom) pausing.

can only be preserved in the bottleneck-equivalent transformation if its usage can be paused by the slowdown mechanism. SLOWPOKE implements the slowdown by pausing the service using POSIX signals [4]. When launching each service, POKER starts first and spawns the service as a child. It also sets a new process group for the service process. To pause a service, POKER first uses an unblockable `SIGSTOP` signal that pauses the service's entire process group. Then the POKER process sleeps for the desired period, calculated as the time the service needs to stop according to the model. Lastly, POKER sends `SIGCONT` to resume the service process.

This implementation correctly preserves general CPU time bottlenecks as well as CPU bottlenecks inside synchronization primitives. It does not accurately preserve network or disk I/O bottlenecks because they have extensive buffering and caching inside the OS kernel.

One notable property of POKER's pausing is that it prevents any CPU usage during the pausing period, even if the service did not fully utilize CPU outside of the period. In contrast, cgroup [25] controls CPU usage by counting actual total CPU time used by the service, and will not slow down the service as SLOWPOKE needs, if the CPU quota was not fully used. As a result, SLOWPOKE can accurately predict throughput when a mutex-induced bottleneck exists in other services in §5.3.

Batching: POSIX signals can have non-negligible overhead due to the need for context switching the processes off the CPUs, especially if the original service has a low per-call processing time. To combat this problem, SLOWPOKE accumulates pauses and executes them together in batches, reducing the overall overhead. We evaluate the overhead of POKER's pausing mechanism and the effect of batching in § 5.4.

Pause-induced latency: One subtle yet important scalability issue hides in the exact choice for when to do the pauses. Suppose we implement pauses in a straightforward way: let POKER pause the local service based on the incoming requests it has seen. This uncoordinated local pausing scheme has a serious scalability problem: the average latency in the profiling phase would increase rapidly as the number of services on the critical path increases.

To illustrate this problem, consider the scenario shown in

Table 2: Benchmark Summary

Benchmark	Services	LOC	Sources
OnlineBoutique	9	1088	[3,53]
HotelReservation	6	608	[21,53]
SocialNetwork	6	532	[21,38,39,53]
MovieReview	12	913	[8,21,53]

Fig. 6. Service A calls service B, which calls service C, and service B is the optimization target. Suppose service A and C will enter the pausing period after seeing another 3 calls. Because pausing the service stops it from making more calls, the propagation of the pause among the services will be essentially serialized, leading to the request experiencing very high latency, proportional to the number of services it goes through. Moreover, the requests that directly follow the aforementioned request will also experience the serialized pauses and high latency, leading to high average latency overall.

In an ideal scenario, this will not lead to throughput degradation because the bubble in service C can be filled by queueing many calls before service A enters the pausing period to keep it busy. However, the unavoidable increase in average latency and, by Little's law, in-flight requests in the system, will lead to more resource consumption (*e.g.*, memory, opened HTTP connections), contention (*e.g.*, scheduling, memory cache), ultimately skewing the results of maximal throughput.

Coordinated pausing: To resolve the problem, POKER coordinates the pause periods among services. Instead of each POKER instance pausing its service individually, the target service will decide when *all* the services should pause. Each POKER instance creates additional control channels with its neighbors. The target service will initiate the pausing by sending pausing messages to its neighbors, and using a gossip protocol, the pause will eventually propagate across the entire application. Whenever a POKER instance receives the slowdown message to pause, it further propagates the slowdown message to all of its directly connected neighbors and pauses the service based on currently batched pause time. This method is preferred over a global controller or pairwise connections because it does not require all-to-all network connectivity, and distributes the propagation among the services, reducing the average overhead it puts on a single service.

The coordination is demonstrated in Fig. 6 (bottom). The pause messages help coordinate the services and reduce the overall latency. We call pause messages spawned from the same initial pause at the target service to be in the same *round*. The pause messages include the round number such that each service does not sleep twice for the same round. Consider all the service calls that a user request can trigger forming a graph: the service call graph. The service calls may be delayed at some services because they are paused by POKER. However, assuming that the slowdown messages do not travel slower than the service calls, any of the paths in the service call graph will not encounter pauses for the same round. This ensures that the extra latency introduced by SLOWPOKE no longer increases with the number of services in the application.

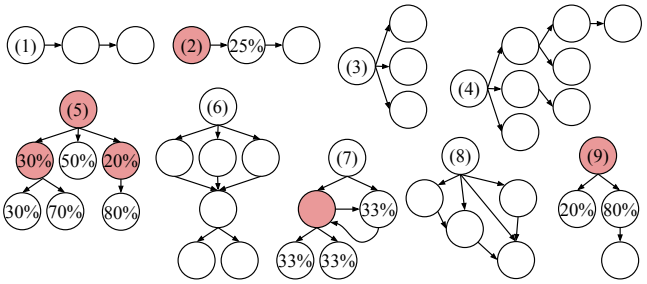


Figure 7: Each block represents a service, with arrows denoting call dependencies between services. Red blocks indicate services that issue dynamic calls, with the probability of a call reaching a callee service specified within the callee block. Topologies of synthetic benchmarks: (1) Chain, (2) Dynamic chain, (3) Fan-out, (4) Unbalanced tree, (5) Dynamic tree with a two-level dynamic path, (6) Directed Acyclic Graph (DAG) with a relaying service, (7) Dynamic path with cycles (cycles occur at the service level by connecting to the same service via different endpoints), (8) DAG with 5 services , and (9) Dynamic tree simulating a cache hit/miss scenario.

We evaluate the effectiveness of coordinated pausing in Section 5.4 and show that it considerably improves average latency compared to uncoordinated pausing, without requiring significantly more in-flight requests when using a closed-loop workload generator.

5 Evaluation

We evaluate SLOWPOKE's prediction accuracy across a diverse set of well-known applications (§5.1) and 108 synthetic benchmarks that capture a broad spectrum of microservice characteristics (§5.2). We further demonstrate the effectiveness of SLOWPOKE in scenarios involving realistic scaling optimizations, mutex-induced bottlenecks, and large-scale microservice deployments on a 45-node cluster (§5.3). Finally, we analyze key design decisions contributing to SLOWPOKE's accuracy and examine its runtime overheads (§5.4).

Experimental setup: We conduct all experiments on Kubernetes v1.29.14 deployed on AWS EC2. Each worker node is an `m5.large` instance with 2 vCPUs (2.5 GHz Intel Xeon Platinum 8259CL), 10 Gbps network bandwidth, 8 GB RAM, and 32 GB GP3 volumes provided by EBS. The control node and the client node—which workload generators run on—are `m5.2xlarge` instances with 8 vCPUs, 32 GB RAM, and 32 GB GP3 volumes. All nodes are running Ubuntu 22.04. The size of clusters varies from 5 to 45 nodes depending on the number of services in the applications. Each service in our experiments occupies a single node due to interference between cgroup resource limits and SLOWPOKE's pausing mechanism, discussed as part of limitations (§7). The target service ini-

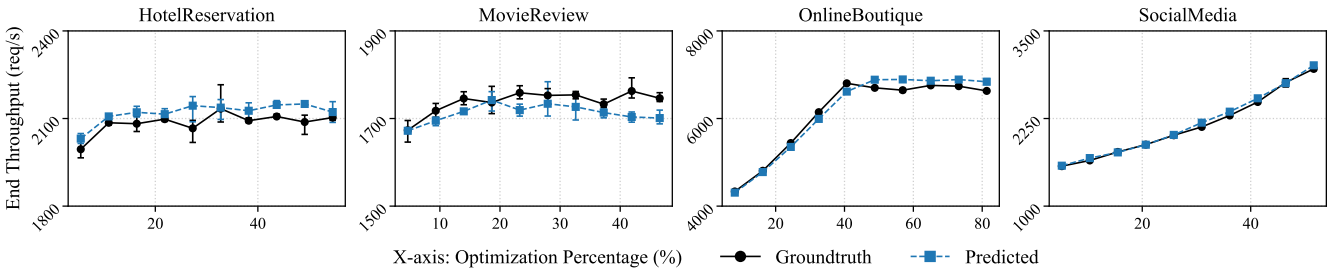


Figure 8: Estimation errors across four well-known applications. The X-axis denotes the optimization percentage of service's processing time, *e.g.*, a 50% optimization corresponds to a 50% reduction in the target service's average processing time per request. The range of percentages varies across applications due to differences in target service's processing times.

tiates slowdown propagation every 100 incoming requests (pause batching size).

Benchmarks: Tab. 2 summarizes four open-source applications used in the evaluation. All services communicate via HTTP. *OnlineBoutique* is an online shopping application and its workload consists of 75% indexing requests (homepage views, product browsing, and cart interactions) and 25% operational requests (currency updates, cart modifications, and checkouts). *HotelReservation* is a hotel booking application and its workload is composed of 80% search queries for hotels in a specific area and 20% reservation requests. *SocialNetwork* is a social media application and its workload distribution includes 60% homepage timeline views, 30% personal timeline views, and 10% post submissions. *MovieReview* is a review-sharing application and its workload consists of 90% page views and 10% review submissions.

To further evaluate SLOWPOKE's accuracy across diverse microservice characteristics, we extend the Hydragen [42] microservice benchmark generator to support dynamic call graphs, expressed as probabilities of invoking downstream services. We generate 9 synthetic topologies shown in Fig. 7 using the extended generator. Each topology is experimentally evaluated with three different configuration parameters: communication protocol (gRPC or HTTP), request concurrency (sequential or concurrent calls), and target service placement (regardless if it is an existing bottleneck), resulting in a total of 108 configurations. Additionally, we generate a large-scale microservice benchmark with a balanced tree topology comprising 43 services to explore SLOWPOKE's scalability (§5.3).

Workload generation: To generate workloads for all benchmarks, we use `wrk`, a closed-loop workload generator, configured with 8 threads and 1024 connections for real-world benchmarks. Given the diversity of synthetic benchmarks, coarse-grained connection settings can overload the system, leading to inaccurate throughput measurements. To address this, we apply a heuristic strategy that reduces the number of connections when the observed throughput is significantly lower than the expected throughput, calculated based on the configured processing times. We apply a consistent workload across all phases of each experiment, including pre-profiling, slowdown execution, and optimization.

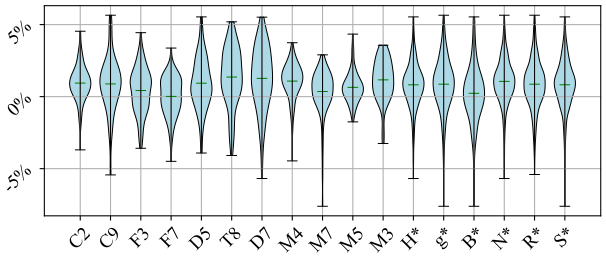
Each throughput measurement includes a 3-second warm-up phase to estimate the expected execution time for the workload, followed by a measurement phase that runs until the workload is completed. We repeat each experiment five times and retain the three data points closest to the median for analysis due to the inherent variance in throughput measurements (§5.1). We calculate the error percentage based on the throughput predicted by SLOWPOKE and the ground truth, which is measured after applying the proposed optimization. Negative errors indicate underestimation, while positive errors indicate overestimation. For a group of predictions, we will report the error range (negative to positive) as well as the root mean squared error to indicate the spread.

5.1 Real-world Microservice Benchmarks

This experiment evaluates SLOWPOKE's prediction accuracy across the four open-source applications in Tab. 2.

Methodology: To perform controlled experiments with varying optimization percentages, we introduce an artificial spinning of 1000 µs to target services and simulate optimizations by removing different fractions of the spinning time. We select target services with varying request loads and simulate optimization by uniformly reducing the spinning time across 10 experiments, resulting in varying optimization percentages as shown in Fig. 8. Specifically, we select `Cart` (accessed by 45% of user requests) in *OnlineBoutique*, `Hometimeline` (70%) in *SocialNetwork*, `Profile` (80%) in *HotelReservation*, and `Moviereviews` (90%) in *MovieReview*. Each benchmark is initialized with predefined deterministic states as in prior work [53], such as users in *SocialNetwork* and reviews in *MovieReview*, prior to execution.

Key results: Fig. 8 illustrates the prediction accuracy of SLOWPOKE across all applications, with error percentages ranging between -4.35–4.88% (R: 2.07%). In *OnlineBoutique*, SLOWPOKE achieves errors ranging between -2.88–4.25% (R: 2.38%). For *SocialNetwork*, errors range between -2.19–2.89% (R: 1.74%). *HotelReservation* shows errors ranging between -2.29–4.88% (R: 2.16%). In *MovieReview*, SLOWPOKE



C2—Chain (length 2) T8—Unbalanced tree M3—Dynamic cache
C9—Chain (length 9) D7—DAG (7 services) H*—HTTP, g*—gRPC
F3—Fan-out (width 3) M4—Dynamic chain B*—target is bottleneck
F7—Fan-out (width 7) M7—Dynamic tree N*—target is not bottleneck
D5—DAG (5 services) M5—Dynamic cycle R/S*—concurrent/sequential

Figure 9: Prediction errors for synthetic experiments grouped by different characteristics. The green lines indicate the mean.

achieves errors ranging between -4.35–1.35% (R: 1.95%).

Analysis: The results demonstrate that SLOWPOKE generalizes to diverse applications, accurately predicting throughput improvements with reasonable errors, given the inherent variance in throughput measurements. For example, the standard deviation of ground truth (without slowdowns) throughputs measured across three runs per configuration is up to 58 req/s (mean: 2134 req/s) for *HotelReservation*. High errors may stem from SLOWPOKE's runtime overheads (see §5.4) and Eq.9's numerically amplifying errors on measured t_{vs}.

Overall, SLOWPOKE accurately predicts performance improvements across a wide range of throughput optimization levels. It predicts gains between 1565–3000 req/s in *SocialNetwork*, and between 4301–6824 req/s in *OnlineBoutique*, when both target services are critical bottlenecks. It also effectively identifies cases where optimizations yield little benefit in the case of *MovieReview* and *HotelReservation* benchmarks.

5.2 Synthetic Benchmarks

In this experiment, we use synthetic benchmarks to evaluate the impact of diverse microservice topologies and configurations on SLOWPOKE's prediction accuracy.

Methodology: We use the 9 topologies of Fig. 7 to generate 108 synthetic benchmarks with varying configurations, including different communication protocols, request concurrency models, and target service placements. For example, in the chain topology, selecting the head, middle, or tail service as the target results in three distinct configurations. All processing times are randomly generated from a Gaussian distribution with a mean of 700 µs and a standard deviation of 300 µs. For each configuration, we generate predictions for optimization ranging from 10% to 100% of the target service's processing time, yielding 108 × 10 experiments in total. Ground truth is obtained by directly modifying the processing time—an input field to the benchmark generator.

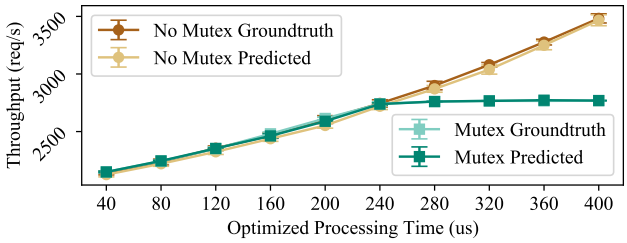


Figure 10: Throughput Prediction with Mutex Bottleneck

Key results: Across configurations, SLOWPOKE's errors range between -7.61–5.65% (R: 1.89%). SLOWPOKE predicts throughput improvements across diverse microservice topologies and configurations with high accuracy, achieving error ranges comparable to those observed in real-world applications. The heatmap shows that underestimations are more common at higher optimization percentages, *e.g.*,-7.61% at a 90% optimization, where the target service's processing time approaches zero, creating an unrealistically fast service. As a result, the throughput measured during slowdown execution is lower due to profiling overhead, resulting in underestimation. Full results for every configuration are shown in Appendix A.

Analysis: To examine the impact of different topologies and configurations on prediction accuracy, we aggregate all results based on topology and configuration parameters, including communication protocol, request concurrency model, and if the target service is an existing bottleneck, as shown in Fig. 9. Out of 1080 experiments, 300 experiments profile the target service as an existing bottleneck, 382 experiments result in underestimation, 540 experiments use sequential calls, and 540 experiments use HTTP protocol. SLOWPOKE maintains high prediction accuracy across topologies, with slightly larger error ranges in scenarios involving more services and higher complexity, *e.g.*, the dynamic topology where multiple dynamic paths are exercised.

5.3 SLOWPOKE in Action

This experiment evaluates SLOWPOKE's prediction accuracy in three scenarios involving realistic optimizations, mutex-induced bottlenecks, and large-scale deployments.

Optimization by scaling: This experiment explores SLOWPOKE's effectiveness in predicting throughput improvements given realistic optimizations including horizontal and vertical scaling. For horizontal scaling, we scale `Cart` in *OnlineBoutique* from 1 to 2 replicas using an additional EC2 machine of the same type. For vertical scaling, we scale `Hometimeline` in *SocialNetwork* from 2 to 4 CPUs by deploying the optimized service on an `m5.xlarge` instance with 4 vCPUs. SLOWPOKE requires an expected optimization effect—by how much the throughput of the target service improves after the optimization—as input to predict the end-to-end throughput improvement. We assume a linear benefit for horizontal

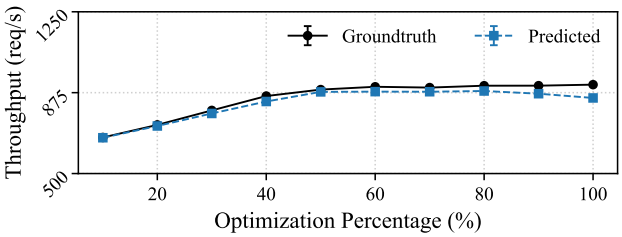


Figure 11: Large-scale deployment with 43 services.

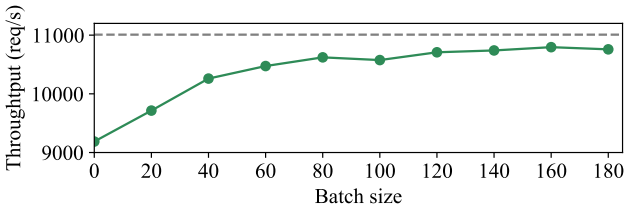


Figure 12: Throughput of a single service with increasing batch size.

scaling for simplicity. Given that vertical scaling does not lead to linear capacity improvement in practice, we first estimate the throughput improvement of `Hometimeline` on 4 vCPUs by provisioning 8 vCPUs to other services to saturate `Hometimeline` at best effort. This value is estimated and used as an input to SLOWPOKE in every slowdown experiment. The results show that SLOWPOKE can accurately predict the throughput improvement of both optimizations, with errors of **-2.41%** for *OnlineBoutique* and **-4.60%** for *SocialNetwork*.

Handling mutex-protected resources: This experiment explores if SLOWPOKE's slowdown mechanism can correctly predict the throughput of an optimized application even when the new bottleneck of the application is a mutex contention. We use a simple two-service application where each service exposes a single endpoint. The client calls `A` and `A` calls `B`. Each service runs on a 2-CPU machine. Inside its endpoint of `A`, each request spins the CPU for 800 microseconds. In the endpoint of `B`, each request spins the CPU for 350 µs inside a mutex. Then we evaluate SLOWPOKE's effectiveness on 10 different optimizations: processing time reduction between 0 and 400 µs. For comparison, we also repeat the two experiments above with the mutex for `B` removed. Fig. 10 presents the actual and predicted throughput in both the mutexed and un-mutexed versions of `B`. Because both `A` and `B` run on 2 CPUs, but `B`'s majority processing time is serialized in a lock, the mutex-ed version of the application starts by gaining performance as `A` is optimized, but then plateaus at 2750 req/s, due to hitting the bottleneck in `B`. For comparison, when `B` is not mutex-ed, `A` remains the bottleneck across the experiments, so the post-optimization throughput keeps increasing. In both cases, SLOWPOKE predicts the throughput with <1% error across all optimizations, demonstrating that its slowdown mechanism works properly with synchronization-induced bottlenecks.

Large-scale deployment: This experiment explores SLOWPOKE's effectiveness in a large-scale deployment with 43 services using a synthetic balanced tree topology (width-6, height-3). Each service's processing time is randomly assigned as in prior synthetic experiments. We choose `service4` at level 2—having both parent and child services—as the optimization target, and apply 10 optimizations reducing its processing time.

Fig. 11 shows SLOWPOKE's prediction in this setup, yielding an average error of -2.50%, ranging between -7.11—0.12% (R: 1.79%). The largest error occurs under extreme optimizations, where the processing time approaches zero. This experiment demonstrates that SLOWPOKE can accurately predict throughput improvements in large-scale deployments.

5.4 Batching and Coordinated Pausing

This set of experiments evaluates SLOWPOKE's batching and coordinated slowdown mechanism.

Pause batching: We first evaluate the overhead of SLOWPOKE as we vary the batching size using a simple two-service call graph. The user-facing service accepts HTTP requests, and calls the inner service, which executes a busy-spinning loop for 50 µs, then returns. This application's throughput, without any instrumentation, is 10960 reqs/s. We use POKER to add a no-op slowdown to the service, meaning that it sends the signals but the sleep time is set to 0*s*. As shown in Fig. 12, we observe a service throughput of 9180 reqs/s (19.4% overhead) without batching. We then increment the batching size by steps of 20, observing throughput gradually increase to 10,795 req/s (1.93% overhead).

Pause coordination: To demonstrate the necessity of pause coordination in SLOWPOKE, we create three chain topologies, with 3, 6, and 9 services, each service deployed on a 2-CPU machine. The root service takes 1ms processing time per request, inner services have processing times selected at random from the range of 100–150 µs. We select the root service as the target and measure the throughput for a hypothetical optimization that reduces its processing time by 600 µs. Coordinated slowdown is implemented by the mechanism specified in Section 4, with a batch size of 50 requests; uncoordinated slowdown is implemented by counting received requests locally and sleeping on every 50 requests.

The first experiment is to evaluate the impact on prediction accuracy using the chain of 9 services. We fix the number of connections to 128 for the coordinated pause and gradually increase the connections to 352, which maximizes the measured throughput, for the uncoordinated pause. The results in Tab. 3 show that SLOWPOKE can predict the end-to-end throughput with a -4.3% error, while uncoordinated pausing,

Table 3: Prediction accuracy comparison between SLOWPOKE's coordinated pause and uncoordinated pause. The ground truth throughput with 128 connections is 3541 req/s.

Setting (conns)	Tput.	Pred.	Rel. error
SLOWPOKE (128)	1680	3387	-4.35%
Uncoordinated pause (128)	956	1340	-62.2%
Uncoordinated pause (352)	1562	2939	-17.0%

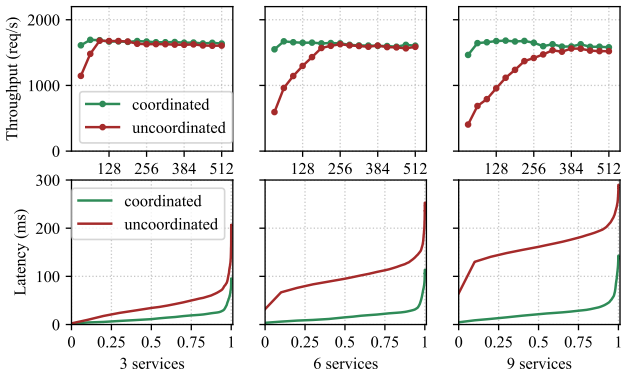


Figure 13: **Top row:** Throughput measured with varying connection parameter in `wrk`. The applications have a simple chain topology with 3, 6, and 9 services. **Bottom row:** Latency at different percentiles for the same set of applications.

even with a high number of connections, results in a -17% prediction error.

To further illustrate the root cause of the problem, we measure all three applications' throughput with different connection numbers (32–512), with results shown in the top of Fig. 13. Without coordinated pausing, the application takes more open connections to reach the peak throughput. More specifically, it needs 96, 256, and 352 open connections for 3, 6, 9 services in the application respectively. With coordinated pause, SLOWPOKE is able to reach peak throughput with no more than 128 open connections when profiling all three topologies. Next, we use an open-loop workload generator, with request rate set at 1500 req/s, and record the latency distribution. As shown in Fig. 13, the 90%ile latency remains stable with coordinated slowdown, demonstrating the efficiency and scalability of the synchronization. The 99%ile latency is slightly higher when increasing the number of services, which is expected as the coordination is not a strict synchronization, which is infeasible in practice.

6 Related Work

SLOWPOKE is related to a large body of prior work on modularity, distributed tracing, critical path analysis, microservice resource management, and service meshes.

Distributed tracing: Distributed tracing [2, 9, 19, 45] techniques have been extensively explored to gain visibility and diagnose performance issues in distributed systems. Pivot Tracing [33], Canopy [30], DeepFlow [43], 3MileBench [54], Hindsight [55], and Black-box performance analysis [10] extend distributed tracing for visibility in diverse environments and under different settings.

The core difference between these approaches and SLOWPOKE is that, traces alone can only capture service dependencies and requests' time spent in each service, whereas what-if questions of the form answered with SLOWPOKE require accurate modeling that captures the complex service dependencies and behaviors after optimizations. That said, SLOWPOKE can leverage tracing information to extract necessary dependency information required for its analysis.

Critical path analysis: Critical path analysis aims to identify latency bottlenecks in complex distributed applications. Mystery Machine [16] and LatenSeer [57] reconstruct dependency relations based on tracing information, identify critical paths for latency, which can be used for what-if analysis of end-to-end latency. CRISP [58] introduces a critical path framework for performance debugging in distributed systems, leveraging trace analysis to pinpoint sources of latency. SnailTrail [26] generalizes critical path analysis for online analysis of long running dataflow. What-if analysis for latency may enable throughput prediction if all the services are CPU-bound and handle requests in a sequential and synchronous way, but not in the general cases that SLOWPOKE handles.

Modularity and profiling: Modularized applications predate the advent of microservices, as exemplified by the staged event-driven architecture [51]. Transactional profiling [14] tracks transactions across shared memory, events, or stages, and measures their interference. Active dependency discovery [13] perturbs system behavior to reveal dynamic dependencies between components. Intra-box analysis [23] introduces variable delays on network and disk traffic to confirm causal relationships between events. While effective in characterizing complex dependencies, these approaches do not quantify the potential gains of optimizations.

Performance modeling: Blocked-time analysis [36] uses detailed white-box logging and full execution replay to quantify I/O bottlenecks. Indy [24] predicts process performance on a single machine by tracing resource usage and calculating predictions based on available hardware resources. These systems predict potential system performance, yet require white-box visibility into resource usage, whereas SLOWPOKE implicitly takes them into account within experiments. Causal profilers [12, 52]—targeting parallel programs on multiple nodes—fail to directly and effectively apply to long-running microservices, due to their different execution models that involve multiple concurrent requests.

Microservice resource management: Microservice resource management focuses on allocating resources efficiently while minimizing latency and cost. FIRM [37], Erms [32], and Sinan [56] use learning-based techniques to efficiently

provision resources to meet application-level latency requirements. Accelerometer [46] provides fine-grained profiling to identify resource usage hotspots in microservice environments. Autothrottle [50] uses throttle measurement to maximize efficiency of CPU provisioning in the application while meeting performance targets. Kraken [49] uses live traffic testing to help stress production systems with realistic workloads. While these systems can be useful in identifying performance bottlenecks, they can neither assist in what-if analysis nor quantify the benefits of hypothetical optimizations.

Service mesh: Existing production-grade service meshes such as Istio [6], Linkerd [7], and Cilium [5] offer robust platforms for managing microservices, featuring traffic management, security, and observability. MeshInsight [60] enhances performance analysis by profiling service meshes through inter-service communication metrics and network latency. While these systems provide valuable insights and actionable capabilities, they do not directly leverage this information to guide optimization planning. SLOWPOKE can potentially be integrated with service meshes and take advantage of their rich observability and traffic control features.

7 Discussions and Limitations

Even though we have shown that SLOWPOKE is effective in many realistic scenarios, we acknowledge that its current mechanism may not be applicable to all possible use cases encountered in production microservice deployments. In this section, we discuss SLOWPOKE's limitations and possible extensions we leave for future work.

Supporting resource sharing between services: The SLOWPOKE performance model assumes exclusive resource usage by each service. However, in cloud environments, it is often beneficial for multiple services to share resources to improve utilization and reduce costs. Future work could extend the model to support resource sharing by introducing constraints:

$$\left(\sum_s \frac{u_s c_s}{q_i} + \frac{c_1 d}{q_1}\right) t \leq 1$$

where u_s represents the resource usage of service s, c_s denotes the service call ratio, and q_i is the total quota allocated to the shared resource. To accommodate this extension, the slowdown mechanism needs to coordinate pauses across all services sharing the resource.

Supporting other applications: SLOWPOKE's current implementation contains a runtime component in Go, embedded within the communication libraries. But the model (§3) is language-agnostic. To support more application/services developers can port the runtime to language/framework where it instruments layer-7 communications. Such implementation extension will enable a broader range of complex microservice applications to use SLOWPOKE.

Compatibility with I/O-bound bottlenecks: The performance model generalizes across various types of bottlenecks, including CPU contention, mutex contention, and network saturation. However, its slowdown mechanism, implemented by `SIGSTOP` signal, has limited effects on the OS kernel. For instance, once a network packet is pushed into the kernel, `SIGSTOP` cannot prevent further kernel and NIC processing. For future work, we believe network bottlenecks could be addressed by using a side car (*e.g.*, Istio [6]) or I/O throttling to slow down network transmission.

Compatibility with cgroup resource limiting: Container orchestration platforms, such as Kubernetes and Docker compose, rely on Linux control groups (cgroup) [25] to enforce resource quotas, if specified. CPU cgroup controls CPU quota by restricting the service's total running time in each consecutive time interval. These time intervals are called "periods", and the quota is refreshed at the beginning of each period. If POKER's pause starts in the middle of a period, the service might have already used up the CPU quota for this period, and would have been stopped anyway. Such interaction can partially or completely nullify POKER's pause, and cause the service to run much faster than SLOWPOKE's model intended. It is worth noting that CPU allocation approaches without such "catch up" mechanisms, like virtual machines and CPU affinity [1], do work with SLOWPOKE. Given that cgroup contains a pausing mechanism `cgroup.freeze` [25], a kernel change that extends the CPU quota refreshing strategy can make cgroup support SLOWPOKE's pausing semantics.

Autoscaling: SLOWPOKE's current model only supports reasoning about services with static resource allocation. Nevertheless, SLOWPOKE can augment autoscaling systems [20, 40]—where resource provisioning is dynamically adjusted—by predicting impacts of optimizations, including both horizontal and vertical scaling.

8 Conclusion

SLOWPOKE accurately quantifies the end-to-end throughput improvements of a hypothetical optimization. By introducing a novel performance model, selective service slowdowns, and an effective coordinated pausing mechanism, SLOWPOKE accurately estimates these effects as demonstrated on microservice applications deployed on AWS.

Acknowledgements

We thank the NSDI'26 reviewers for their feedback; our shepherd, Suman Nath, for his guidance; the NSDI'26 Artifact reviewers for their time. We also thank Chameleon Cloud for providing resources; Vivek Unnikrishnan for starting the project; Devon Lewis for setting up the initial infrastructure; Yuhang Song, Evangelos Lamprou, and Zekai Li for helping with the artifact evaluation. This material is based upon research supported by NSF awards CNS-2247687, CNS-2312346, CNS-2237193, and CNS-2440334; DARPA

contract no. HR001124C0486; a Fall'24 Amazon Research Award; a Google ML-and-Systems Junior Faculty award; and a BrownCS Faculty Innovation Award.

References

[1] CPU Affinity. https://www.gnu.org/software/libc/manual/html_node/CPU-Affinity.html. [Online; accessed April 2025].

[2] Jaeger: Open source, end-to-end distributed tracing. https://www.jaegertracing.io. [Online; accessed April 2025].

[3] Online boutique benchmark. https://github.com/GoogleCloudPlatform/microservices-demo.

[4] signal(7) — Linux manual page. https://man7.org/linux/man-pages/man7/signal.7.html. [Online; accessed April 2025].

[5] The eBPF-powered Cilium service mesh. https://cilium.io/blog/2021/12/01/cilium-service-mesh-beta/. [Online; accessed April 2025].

[6] The Istio service mesh. https://istio.io. [Online; accessed April 2025].

[7] The Linkerd service mesh. https://linkerd.io. [Online; accessed April 2025].

[8] The Movie Database (TMDB) — themoviedb.org. https://www.themoviedb.org/. [Accessed 24-03-2025].

[9] Zipkin: A distributed tracing system. https://zipkin.io. [Online; accessed April 2025].

[10] Marcos K. Aguilera, Jeffrey C. Mogul, Janet L. Wiener, Patrick Reynolds, and Athicha Muthitacharoen. Performance debugging for distributed systems of black boxes. In *Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles*, SOSP '03, page 74–89, New York, NY, USA, 2003. Association for Computing Machinery.

[11] Emre Baran. Performance and scalability of microservices. https://www.cerbos.dev/blog/performance-and-scalability-microservices. [Online; accessed April 2025].

[12] Zachary Benavides, Keval Vora, and Rajiv Gupta. Dprof: distributed profiler with strong guarantees. *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):1–24, 2019.

[13] Aaron Brown, Gautam Kar, and Alexander Keller. An active approach to characterizing dynamic dependencies for problem determination in a distributed environment. In *2001 IEEE/IFIP International Symposium on Integrated Network Management Proceedings. Integrated Network Management VII. Integrated Management Strategies for the New Millennium (Cat. No. 01EX470)*, pages 377–390. IEEE, 2001.

[14] Anupam Chanda, Alan L Cox, and Willy Zwaenepoel. Whodunit: Transactional profiling for multi-tier applications. In *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems 2007*, pages 17–30, 2007.

[15] Michael Chow, David Meisner, Jason Flinn, Daniel Peek, and Thomas F. Wenisch. The mystery machine: End-to-end performance analysis of large-scale internet services. In *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation*, OSDI'14, page 217–231, USA, 2014. USENIX Association.

[16] Michael Chow, David Meisner, Jason Flinn, Daniel Peek, and Thomas F Wenisch. The mystery machine: End-to-end performance analysis of large-scale internet services. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 217–231, 2014.

[17] Charlie Curtsinger and Emery D Berger. Coz: Finding code that counts with causal profiling. In *Proceedings of the 25th Symposium on Operating Systems Principles*, pages 184–197, 2015.

[18] Anca Agape Daniel Boeve, Kiryong Ha. Throughput autoscaling: Dynamic sizing for facebook.com, 2020.

[19] Rodrigo Fonseca, George Porter, Randy H. Katz, and Scott Shenker. X-Trace: A pervasive network tracing framework. In *4th USENIX Symposium on Networked Systems Design & Implementation (NSDI 07)*, Cambridge, MA, April 2007. USENIX Association.

[20] Guilherme Galante, Luis Carlos Erpen De Bona, Antonio Roberto Mury, Bruno Schulze, and Rodrigo da Rosa Righi. An analysis of public clouds elasticity in the execution of scientific applications: a survey. *Journal of Grid Computing*, 14(2):193–216, 2016.

[21] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 3–18, 2019.

[22] Adam Gluck. Introducing domain-oriented microservice architecture, 2020.

[23] Haryadi S Gunawi, Nitin Agrawal, Andrea C Arpaci-Dusseau, J Schindler, and RH Arpaci-Dusseau. Deconstructing commodity storage clusters. In *32nd International Symposium on Computer Architecture (ISCA'05)*, pages 60–71. IEEE, 2005.

[24] Jonathan C Hardwick, Efstathios Papaefstathiou, and David Guimbellot. Modeling the performance of e-commerce sites. In *27th International Conference of the Computer Measurement Group*, volume 105, page 3, 2001.

[25] Tejun Heo. Control group v2. https://docs.kernel.org/admin-guide/cgroup-v2.html, 2025.

[26] Moritz Hoffmann, Andrea Lattuada, John Liagouris, Vasiliki Kalavri, Desislava Dimitrova, Sebastian Wicki, Zaheer Chothia, and Timothy Roscoe. SnailTrail: Generalizing critical paths for online analysis of distributed dataflows. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*, pages 95–110, Renton, WA, April 2018. USENIX Association.

[27] Will Hu. Improving performance and capacity for espresso with new netty framework, 2019.

[28] Darby Huye, Yuri Shkuro, and Raja R Sambasivan. Lifting the veil on {Meta's} microservice architecture: Analyses of topology and request workflows. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, pages 419–432, 2023.

[29] Yurong Jiang, Lenin Ravindranath Sivalingam, Suman Nath, and Ramesh Govindan. Webperf: Evaluating what-if scenarios for cloud-hosted web applications. In *Proceedings of the 2016 ACM SIGCOMM Conference*, SIGCOMM '16, page 258–271, New York, NY, USA, 2016. Association for Computing Machinery.

[30] Jonathan Kaldor, Jonathan Mace, Michał Bejda, Edison Gao, Wiktor Kuropatwa, Joe O'Neill, Kian Win Ong, Bill Schaller, Pingjia Shan, Brendan Viscomi, Vinod Venkataraman, Kaushik Veeraraghavan, and Yee Jiun Song. Canopy: An end-to-end performance tracing and analysis system. In *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP '17, page 34–50, New York, NY, USA, 2017. Association for Computing Machinery.

[31] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Yu Ding, Jian He, and Chengzhong Xu. Characterizing microservice dependency and performance: Alibaba trace analysis. In *Proceedings of the ACM symposium on cloud computing*, pages 412–426, 2021.

[32] Shutian Luo, Huanle Xu, Kejiang Ye, Guoyao Xu, Liping Zhang, Jian He, Guodong Yang, and Chengzhong Xu. Erms: Efficient resource management for shared microservices with sla guarantees. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS 2023, page 62–77, New York, NY, USA, 2022. Association for Computing Machinery.

[33] Jonathan Mace, Ryan Roelke, and Rodrigo Fonseca. Pivot tracing: Dynamic causal monitoring for distributed systems. In *2016 USENIX Annual Technical Conference (USENIX ATC 16)*, Denver, CO, June 2016. USENIX Association.

[34] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 303–320, Carlsbad, CA, July 2022. USENIX Association.

[35] Irakli Nadareishvili, Ronnie Mitra, Matt McLarty, and Mike Amundsen. *Microservice architecture: aligning principles, practices, and culture*. " O'Reilly Media, Inc.", 2016.

[36] Kay Ousterhout, Ryan Rasti, Sylvia Ratnasamy, Scott Shenker, and Byung-Gon Chun. Making sense of performance in data analytics frameworks. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 293–307, Oakland, CA, May 2015. USENIX Association.

[37] Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. FIRM: An intelligent fine-grained resource management framework for SLO-Oriented microservices. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 805–825. USENIX Association, November 2020.

[38] Ryan Rossi and Nesreen Ahmed. The network data repository with interactive graph analytics and visualization. In *Proceedings of the AAAI conference on artificial intelligence*, 2015.

[39] Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization. In *AAAI*, 2015.

[40] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, et al. Autopilot: workload autoscaling

at google. In *Proceedings of the Fifteenth European Conference on Computer Systems*, pages 1–16, 2020.

[41] Gerald Schermann, Jürgen Cito, Philipp Leitner, Uwe Zdun, and Harald C Gall. We're doing it live: A multi-method empirical study on continuous experimentation. *Information and Software Technology*, 99:41–57, 2018.

[42] Mohammad Reza Saleh Sedghpour, Aleksandra Obeso Duque, Xuejun Cai, Björn Skubic, Erik Elmroth, Cristian Klein, and Johan Tordsson. Hydragen: A microservice benchmark generator. In *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*, pages 189–200. IEEE, 2023.

[43] Junxian Shen, Han Zhang, Yang Xiang, Xingang Shi, Xinrui Li, Yunxi Shen, Zijian Zhang, Yongxiang Wu, Xia Yin, Jilong Wang, Mingwei Xu, Yahui Li, Jiping Yin, Jianchang Song, Zhuofeng Li, and Runjie Nie. Network-centric distributed tracing with deepflow: Troubleshooting your microservices in zero code. In *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM SIGCOMM '23, page 420–437, New York, NY, USA, 2023. Association for Computing Machinery.

[44] Yuri Shkuro. Conquering microservices complexity @uber with distributed tracing, 2019.

[45] Benjamin H. Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. Dapper, a large-scale distributed systems tracing infrastructure. Technical report, Google, Inc., 2010.

[46] Akshitha Sriraman and Abhishek Dhanotia. Accelerometer: Understanding acceleration opportunities for data center overheads at hyperscale. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '20, page 733–750, New York, NY, USA, 2020. Association for Computing Machinery.

[47] Alexander Tarvo, Peter F Sweeney, Nick Mitchell, VT Rajan, Matthew Arnold, and Ioana Baldini. Canaryadvisor: a statistical-based tool for canary testing. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, pages 418–422, 2015.

[48] Sudhir Tonse. Scalable microservices at netflix. challenges and tools of the trade, 2015.

[49] Kaushik Veeraraghavan, Justin Meza, David Chou, Wonho Kim, Sonia Margulis, Scott Michelson, Rajesh Nishtala, Daniel Obenshain, Dmitri Perelman, and Yee Jiun Song. Kraken: Leveraging live traffic tests to identify and resolve resource utilization bottlenecks in large scale web services. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 635–651, Savannah, GA, November 2016. USENIX Association.

[50] Zibo Wang, Pinghe Li, Chieh-Jan Mike Liang, Feng Wu, and Francis Y Yan. Autothrottle: A practical {Bi-Level} approach to resource management for {SLO-Targeted} microservices. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 149–165, 2024.

[51] Matt Welsh, David Culler, and Eric Brewer. Seda: An architecture for well-conditioned, scalable internet services. *ACM SIGOPS operating systems review*, 35(5):230–243, 2001.

[52] Adarsh Yoga and Santosh Nagarakatte. A fast causal profiler for task parallel programs. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, pages 15–26, 2017.

[53] Haoran Zhang, Konstantinos Kallas, Spyros Pavlatos, Rajeev Alur, Sebastian Angel, and Vincent Liu. {MuCache}: A general framework for caching in microservice graphs. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 221–238, 2024.

[54] Jun Zhang, Robert Ferydouni, Aldrin Montana, Daniel Bittman, and Peter Alvaro. 3milebeach: A tracer with teeth. In *Proceedings of the ACM Symposium on Cloud Computing*, SoCC '21, page 458–472, New York, NY, USA, 2021. Association for Computing Machinery.

[55] Lei Zhang, Zhiqiang Xie, Vaastav Anand, Ymir Vigfusson, and Jonathan Mace. The benefit of hindsight: Tracing Edge-Cases in distributed systems. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 321–339, Boston, MA, April 2023. USENIX Association.

[56] Yanqi Zhang, Weizhe Hua, Zhuangzhuang Zhou, G. Edward Suh, and Christina Delimitrou. Sinan: Ml-based and qos-aware resource management for cloud microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '21, page 167–181, New York, NY, USA, 2021. Association for Computing Machinery.

[57] Yazhuo Zhang, Rebecca Isaacs, Yao Yue, Juncheng Yang, Lei Zhang, and Ymir Vigfusson. Latenseer: Causal modeling of end-to-end latency distributions by harnessing distributed tracing. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*, SoCC '23, page 502–519, New York, NY, USA, 2023. Association for Computing Machinery.

[58] Zhizhou Zhang, Murali Krishna Ramanathan, Prithvi Raj, Abhishek Parwal, Timothy Sherwood, and Milind Chabbi. CRISP: Critical path analysis of Large-Scale microservice architectures. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 655–672, 2022.

[59] Hao Zhou, Ming Chen, Qian Lin, Yong Wang, Xiaobin She, Sifan Liu, Rui Gu, Beng Chin Ooi, and Junfeng Yang. Overload control for scaling wechat microservices. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 149–161, 2018.

[60] Xiangfeng Zhu, Guozhen She, Bowen Xue, Yu Zhang, Yongsu Zhang, Xuan Kelvin Zou, Xiongchun Duan, Peng He, Arvind Krishnamurthy, Matthew Lentz, Danyang Zhuo, and Ratul Mahajan. Dissecting service mesh overheads, 2022.

A Full Results for Synthetic Benchmarks

Fig. 14 shows the error heatmap across all synthetic experiments discussed in the paper (§5.2).

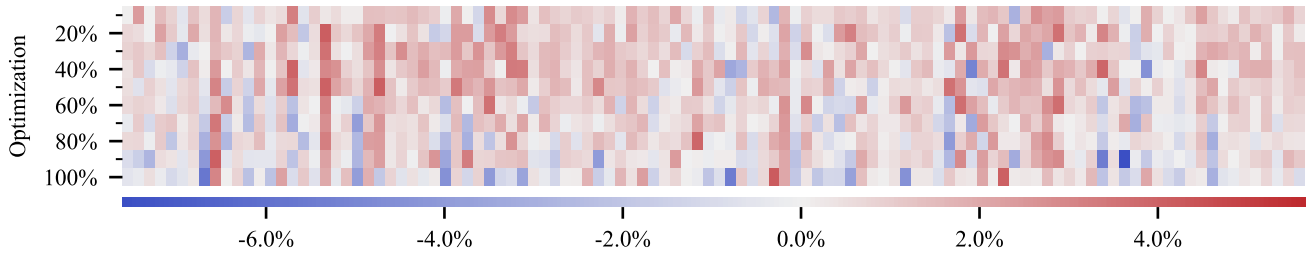


Figure 14: Error heatmap of SLOWPOKE across all synthetic call graph configurations. Each row represents a single callgraph combination, and each cell indicates the estimation error for a specific optimization percentage (10%–100%), where a 50% optimization indicates a 50% reduction of target service's average processing time per request. The intensity of the color corresponds to the magnitude of the error. Red blocks denote over-estimation while blue indicates under-estimation.